

Variables y tipos de datos

Santiago Gallego

2026-05-07

Tabla de Contenido

0.1	Ejercicio: Asignación de variables y operaciones básicas	2
0.1.1	Python	2
0.1.2	R	2
0.1.3	Python	3
0.1.4	R	3
0.1.5	Python	3
0.1.6	R	3
0.1.7	Python	3
0.1.8	R	3
0.1.9	Python	3
0.1.10	R	4
0.2	Ejercicio: Trabajando con Strings	4
0.2.1	Python	4
0.2.2	R	4
0.2.3	Python	4
0.2.4	R	4
0.2.5	Python	4
0.2.6	R	5
0.2.7	Python	5
0.2.8	R	5
0.2.9	Python	5
0.2.10	R	6
0.2.11	Pregunta	6

0.1 Ejercicio: Asignación de variables y operaciones básicas

0.1.1 Python

```
nombre_estacion = "Páramo de Santurban" # Asigna una cadena de texto (string) al nombre
Latitud = 7.2514 # Almacena un número decimal (float) para la latitud
Longitud = -72.9069 # Almacena un número decimal (float) para la longitud
Elevacion_metros = 3350 # Almacena un número entero (int) para la altura
Activo = True # Asigna un valor booleano (True) de estado
```

0.1.2 R

```
nombre_estacion <- "Páramo de Santurban" # Asigna texto a la variable usando el operador <-
Latitud <- 7.2514 # Define la latitud como valor numérico
Longitud <- -72.9069 # Define la longitud como valor numérico
Elevacion_metros <- 3350 # Define la elevación como valor numérico
Activo <- TRUE # Define el estado lógico (TRUE en mayúsculas)
```

0.1.3 Python

```
Coordenadas = [7.2514, -72.9069] # Crea una lista (secuencia mutable) con dos valores
```

0.1.4 R

```
Coordenadas <- c(7.2514, -72.9069) # Crea un vector atómico usando la función de combinar c()
```

0.1.5 Python

```
metadatos_estacion = {                                # Inicia la definición de un diccionario (llave-valor)
    "nombre_estacion": "Páramo de Saturban",
    "Coordenadas": [7.2514, -72.9069],
    "Elevacion_metros": 3350,
    "activa": True
}
```

0.1.6 R

```
metadatos_estacion <- list(                            # Crea una lista etiquetada para almacenar diversos t
    "nombre_estacion"= "Páramo de Saturban",
    "Coordenadas"= c(7.2514, -72.9069),
    "Elevacion_metros"= 3350,
    "activa"= TRUE
)
```

0.1.7 Python

```
metadatos_estacion["Elevacion_metros"] += 12.5 # Suma 12.5 al valor actual mediante acceso por índice
```

0.1.8 R

```
metadatos_estacion$Elevacion_metros <- metadatos_estacion$Elevacion_metros + 12.5 # Suma 12.5 al valor actual mediante acceso por nombre
```

0.1.9 Python

```
Longitud_extraida = metadatos_estacion["Coordenadas"][1] # Accede al segundo elemento de la lista
# Imprime un reporte formateado usando f-string para interpolar variables
print(f"La estacion {metadatos_estacion['nombre_estacion']} se encuentra operativa en la longitud {Longitud_extraida}")
```

La estacion Páramo de Saturban se encuentra operativa en la longitud -72.9069 a una altura a

0.1.10 R

```
longitud_extraida <- metadatos_estacion$Coordenadas[2] # Accede al segundo elemento del vector
# Concatena y muestra el mensaje usando paste0 y cat
cat(paste0("La estación ", metadatos_estacion$nombre_estacion, " se encuentra operativa en la longitud ", longitud_extraida, " a una altura de ", altitud_extraida, " metros"))
```

La estación Páramo de Saturban se encuentra operativa en la longitud -72.9069 a una altura de 2850 metros.

0.2 Ejercicio: Trabajando con Strings

Inicializa una variable con el siguiente texto crudo y problemático:

0.2.1 Python

```
registro_crudo = " sAnTuaRio de fAuna Y flora iGuaQue " # String con espacios y mayúsculas
```

0.2.2 R

```
registro_crudo <- " sAnTuaRio de fAuna Y flora iGuaQue " # Variable con texto sin formato
```

Utiliza el método o función nativa de tu lenguaje (vista en la sección anterior) para eliminar los espacios en blanco accidentales al inicio y al final del texto.

0.2.3 Python

```
registro_limpio = registro_crudo.strip() # Elimina espacios en blanco en ambos extremos del string
print("Registro limpio:", f'"{registro_limpio}"') # Imprime el resultado entre comillas simples
```

```
Registro limpio: 'sAnTuaRio de fAuna Y flora iGuaQue'
```

0.2.4 R

```
registro_limpio = trimws(registro_crudo) # Función "Trim White Space" para limpiar extremos
cat("Registro limpio: '", registro_limpio, "'\n", sep="") # Imprime usando cat sin separador
```

```
Registro limpio: 'sAnTuaRio de fAuna Y flora iGuaQue'
```

Usa el comando correspondiente para convertir el texto limpio a formato de Título (Title Case), estandarizando así las mayúsculas y minúsculas.

0.2.5 Python

```
registro_limpio = registro_limpio.title() # Convierte la primera letra de cada palabra a may
print(f'"{registro_limpio}"') # Muestra el texto transformado
```

'Santuario De Fauna Y Flora Iguaque'

0.2.6 R

```
registro_limpio <- tools::toTitleCase(tolower(registro_limpio)) # Pasa a minúsculas y luego
cat(registro_limpio, "\n", sep="") # Muestra el texto procesado
```

Santuario De Fauna y Flora Iguaque'

Emplea la herramienta de reemplazo de texto para cambiar la letra " Y " por " y " (en minúscula), de modo que el conector gramatical sea correcto.

0.2.7 Python

```
registro_limpio = registro_limpio.title() # Convierte la primera letra de cada palabra a may
print(f'"{registro_limpio}"') # Muestra el texto transformado
```

'Santuario De Fauna Y Flora Iguaque'

0.2.8 R

```
registro_limpio <- tools::toTitleCase(tolower(registro_limpio)) # Pasa a minúsculas y luego
cat(registro_limpio, "\n", sep="") # Muestra el texto procesado
```

Santuario De Fauna y Flora Iguaque'

Utiliza caracteres de escape (\n y \t) para imprimir un pequeño reporte estructurado que muestre el antes y el después

0.2.9 Python

```
print("\n--- REPORTE DE LIMPIEZA DE TEXTO ---") # \n genera un salto de línea inicial
```

```
--- REPORTE DE LIMPIEZA DE TEXTO ---
```

```
print("\tAntes (crudo) :", f'"{registro_crudo}"') # \t inserta una tabulación horizontal
```

```
Antes (crudo) : ' sAnTuaRio de fAuna Y flora iGuaQue '
```

```
print("\tDespués (limpio):", f"{registro_limpio}") # Muestra el resultado final tabulado
```

```
Después (limpio): 'Santuario De Fauna Y Flora Iguaque'
```

```
print("\nProceso completado correctamente ") # Salto de línea y mensaje de éxito
```

```
Proceso completado correctamente
```

0.2.10 R

```
cat("\n--- REPORTE DE LIMPIEZA DE TEXTO ---\n") # \n realiza saltos de línea al inicio y fin
```

```
--- REPORTE DE LIMPIEZA DE TEXTO ---
```

```
cat("\tAntes (crudo) : '", registro_crudo, "'\n", sep = "") # \t para sangría de párrafo
```

```
Antes (crudo) : ' sAnTuaRio de fAuna Y flora iGuaQue '
```

```
cat("\tDespués (limpio): '", registro_limpio, "'\n", sep = "") # Imprime la variable limpia
```

```
Después (limpio): 'Santuario De Fauna y Flora Iguaque'
```

```
cat("\nProceso completado correctamente \n") # Salto de línea final para orden
```

```
Proceso completado correctamente
```

0.2.11 Pregunta

En términos de geoprocésamiento de bases de datos relacionales, ¿por qué es un paso crítico y obligatorio aplicar métodos de limpieza como `.strip()` o `trimws()` antes de realizar uniones de tablas (Joins) basadas en nombres de municipios o regiones?

Porque los joins por strings son frágiles por naturaleza. Un solo espacio invisible puede hacer que se pierdan decenas o cientos de registros, lo que contamina todo el análisis posterior (estadísticas, mapas, cruces espaciales, etc.).